CGRA259 - Caylum Blenkhorn - 300501291

GAME REPORT 3 (Group & Individual)

~~ Part 1: Group ~~

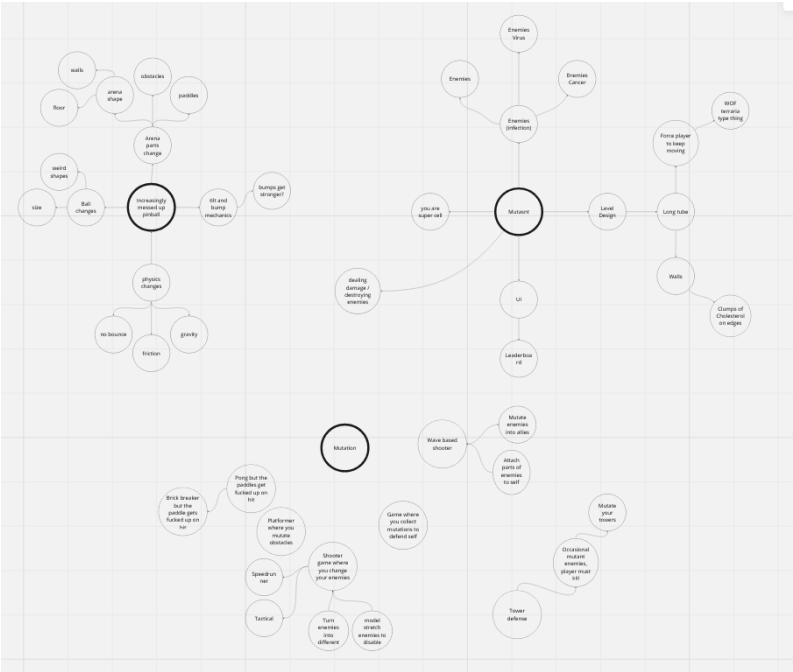
To preface, our group is comprised of myself (Caylum) and a few of my friends (Simon, Jamie, and new to the Jam concept, Thomas), who all are not enrolled in CGRA259 course.

The Third and final jam our group participated in was the 28th **ScoreSpace Jam**, this jam ran from January 27th 10am to the 30th of January 10am, a combined time available to develop our game being 72 hours. The theme of this final jam was **'Mutation'**, with the possible secondary optional theme being **'Momentum'**. Similar to the previous jam, that being 72 hours, we chose to follow a similar timeframe, 1-2 hours to develop our concept, then having the MVP ready to go for submission 24 hours ahead of the end of the Jam.

We began with a few different concepts for a game featuring the Mutation theme:

- A tower defence where the towers are mutated.
- Pinball, except on each ball hit, whatever is hit is mutated into a tumorous/vegetational growth.
- A single stick auto-shooter where the player is a white blood cell defending against germs.

Like the previous Jams, we believe the timeline we set was followed well, having a concept ready to go that we all felt passionate about drastically helped speed up the early stages of the Jam.



For resources, our group used the most recent version of Unreal Engine, 5.3.2, GitLab and Github Desktop (+ gitfls) for version control, a continued use of Javascript and HTML for the web UI elements. We used Piskel, Paint and Gimp for art, and finally as we had a new member in our group who wanted to try and make music/sfx for our game, we also used Tracktion Waveform 12. In engine, we used blueprints for all our coding.

We voted to use 2d graphics for the game as it fit the theme and setting best, with a continued use of piskel/paint to make quick pixel art assets and sprites.

Our group allocated roles:

Caylum: Art, Enemy spawn director

Simon: Web, UI, Leaderboard

Jamie: Art, Player, Enemies

Thomas: Audio

Like the previous Jam, roles were allocated simply by what each group member wanted to work on most, ensuring each member was passionate about what they were working on. We felt this was a good decision to continue with, as it worked well prior.

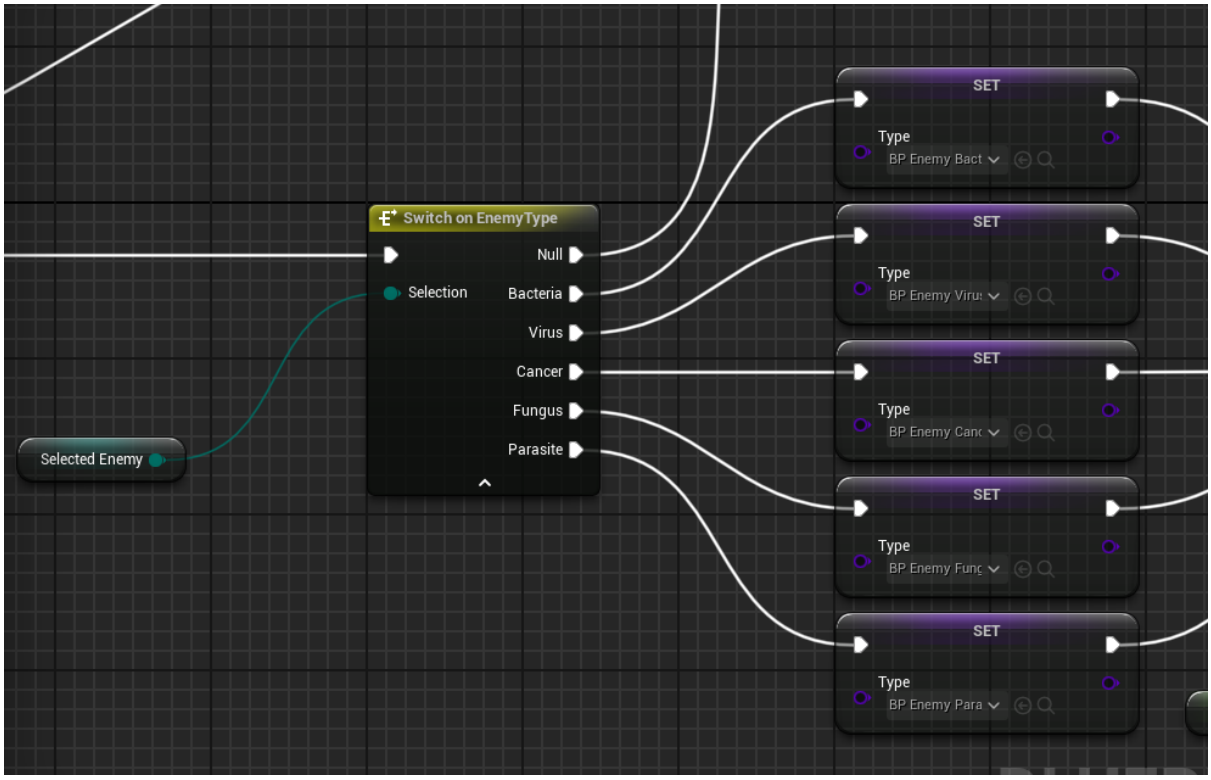
Before we get into the gameplay look and systems at work, it is important to note that for this jam we were required to implement the jam sponsors leaderboard system, LootLocker.

Like other single stick shooters, autoaim is employed so the player does not need to aim, and instead focuses more on movement and dodging. The loop is comprised of short unique games where the player moves around shooting germs and bacteria honing in. The player is able to pick up dropped DNA pegs and slot them into themselves in pairs, and develop new skills depending on the peg combinations, multiple stacks of the same combination increase the efficacy of the skill.

The player restricted to a narrow gameplay area vertically, and is forced to slowly move to the right as a wall travels east, pushing the player.

The enemies consist of 5 different types, and are spawned using a special system to make each game feel unique and have slowly scaling difficulty. Over time the enemy spawn controller gains points in which it can spend to spawn enemies at varying times, it, depending on a level of randomness, can choose to spend or save it's points, and with sufficient points it can spawn larger versions of the regular enemies. To ensure difficult enemies are not spawned too early, each enemy has its own specific point cost, and minimum time it can be selected by the spawn controller.

Additionally, the game uses the previous Jam's implementation of the custom web UI.



The 5 enemies that can be chosen, branching off via and Enumerator and switch case.

~~ Part 2: Individual ~~

1) Teamwork

communication in the group was very effective and consistent, we were in a discord call for the majority of the jam, and frequently conversed to each other about where we were at, what needs attention, and give help if needed.

Tasks were allocated through us all asking what each member wanted to, a member who is forced to work on something they aren't interested in I believe is detrimental to fast success, so each member chose something they had interest or passion in. In addition to having tight communication, help was readily available, and tasks could be swapped around if a member found themselves in a roadblock or got stuck.

Team member strengths also played a role in task allocation, each member working on tasks they were familiar in or with was very helpful.

In addition, this time we tried to be very flexible in tasks and allocation, when one person got stuck or lost interest from fatigue, we would postpone that task and get something else started and anyone could repick up that stuck task.

2) **Individual**

Aside from the art, my main contribution to the jam was through the spawn controller lightly detailed above, the controller goes through many steps in determining what will be spawned, below is a simplified step process of how it functions:

SETUP

1) If spawn timer = 0, start setup
2) If more than 50 enemies alive -> FAIL
3) If enemy type selected and no FAIL, prepare wave (or)
4) If no enemy type selected (null type) or previous spawn attempt was FAIL, set new random enemy.
5) Reset spawn counter, start at 0, increment on this wave's enemies
6) Proceed to TYPE CHECK

TYPE CHECK

1) Does current wave have 5 or less enemies?
2) Is selected enemy valid? (minimum wave, cannot spawn early)
3) Controller can afford this enemy?
4) If all TRUE, proceed to SPAWN, else FAIL and proceed to STATUS

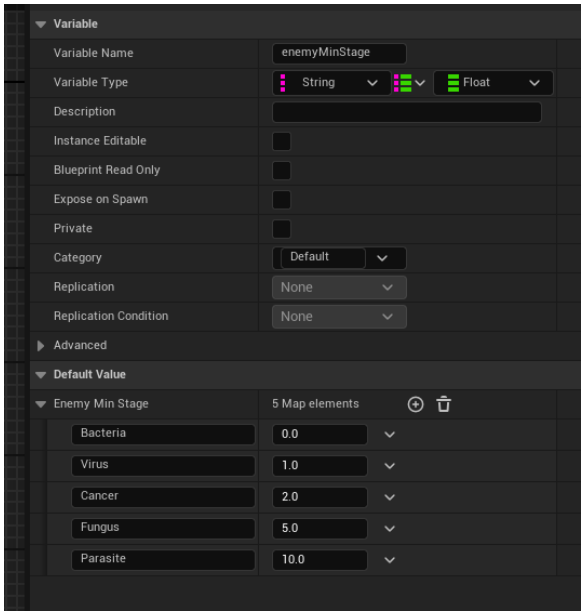
SPAWN

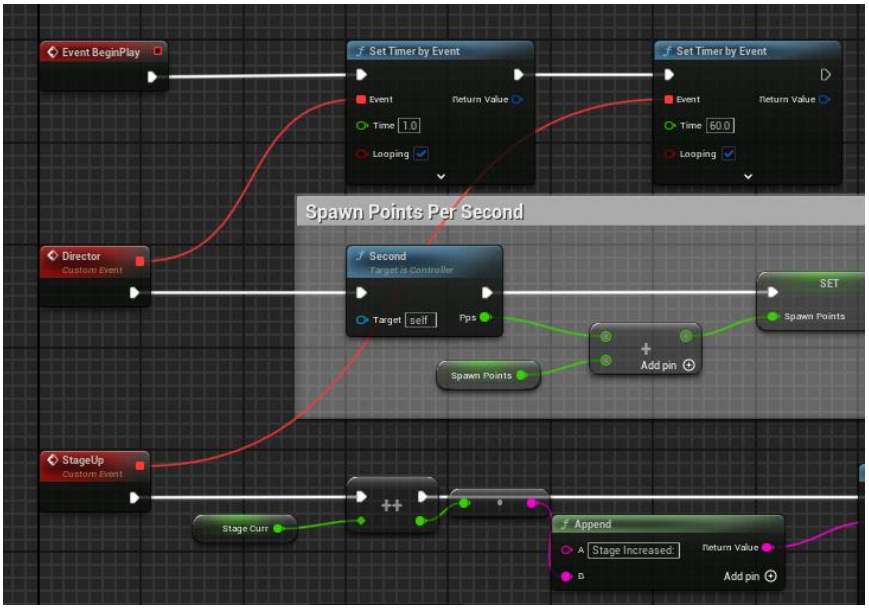
1) Pay for the enemy, reduct points accordingly.
2) If excess points are available, spawn large enemy.
3) Spawn enemy.

STATUS

- On FAIL
    - Increase spawn timer as punishment, if aggressive controller, gain bonus credits
    - Reset selected enemy
- On SUCCESS
    - Increase spawn timer slightly
    - Keep same selected enemy
- Return to SETUP

This controller is always unique each time, feels dynamic to play against, and has increasing difficulty that can be fine tuned to fit the player's scaling.





Left: the enemy minimum waves in a String/Float map

Right: points increasing every second, stage count increasing every minute

3) Changes

While we aimed to have an MVP complete 24 hours before the deadline, we were unable to get this goal completed, the game was not in a very finished state when submitted, but is still mostly playable all things considered. We believe this is due to us trying to incorporate too many systems in the game and having an MVP too large for the timeframe.

On a positive note, our game finally has music and SFX thanks to our new group member Thomas, which was a very nice change, and was also interesting to learn how sounds are played in blueprints.